Deterministic Font-Encoding Repair for Nepali–English Code-Switched Retrieval in Legal RAG Pipelines

Pankaj Adhikari
School of Software Engineering
Nankai University
Tianjin, China
codepariwar@gmail.com

***Abstract*—Local government documents in Nepal frequently suffer severe semantic degradation when parsed by modern layout extractors, because legacy non-Unicode fonts (e.g., Preeti) surface as corrupted ASCII strings such as ``sf7df8f}F'' (which encodes *Kathmandu Metropolitan City*). This paper presents an offline-first hybrid extraction and retrieval pipeline for Nepali–English code-switched legal search, and isolates *font-encoding repair* as the decisive preprocessing step. A deterministic document router dispatches text-layer PDFs to PyMuPDF and scanned PDFs to Surya OCR; extracted text is segmented with heading-aware chunking (512-token window, 128-token overlap fallback). Over a curated corpus of 50 municipal and federal legal PDFs yielding 446 retrieval chunks—of which 26 documents are Preeti-corrupted—sparse TF-IDF retrieval on raw corrupted text attains only Recall@3 = 0.091 (MRR = 0.030). We introduce an open, deterministic Preeti→Unicode repair module (glyph substitution plus vowel-sign and reph reordering) that recovers 45.8% token-level validity against an independent clean-document vocabulary and, applied before indexing, lifts sparse retrieval to Recall@3 = 0.364 and document-level Recall@3 = 0.455—a four- to fivefold gain with *no* hand-authored aliases. We contrast this against a metadata-aliasing configuration that reaches Recall@3 = 1.000 but which we report explicitly as an *oracle upper bound*, since its injected keywords paraphrase the evaluation queries. The residual gap between deterministic repair and the oracle is attributable to conjunct lossiness, motivating learned transliteration repair as future work.**

***Index Terms*—Retrieval-Augmented Generation, Code-Switched NLP, Font Encoding, Preeti, Nepali Legal IR, Document Intelligence, Sparse Retrieval.**

I. Introduction

The digitalization of municipal governance under Nepal's federal structure has produced a large volume of unstructured, low-resource-language PDFs hosted across Kathmandu, Lalitpur, and Bhaktapur portals, as well as federal statute repositories. These assets remain difficult to query with conventional Large Language Models (LLMs): citizens typically ask in Nepali–English code-switched language (e.g., parking fees, property-tax slabs, business-registration documents), while the underlying PDFs often use legacy non-Unicode fonts. When extracted by standard text scrapers, glyphs map to randomized ASCII, destroying subword tokenization and inducing embedding-space mismatches for dense retrievers.

Scope. This paper evaluates the *retrieval substrate* of a Retrieval-Augmented Generation (RAG) system—the stage that determines whether a generator can be grounded at all. Generation quality is discussed qualitatively (Section V-A) but is not the object of quantitative claims here.

We address the extraction gap with a privacy-preserving, edge-deployable pipeline that (i) routes documents by curated format type, (ii) chunks with legal heading awareness, (iii) *deterministically repairs Preeti-encoded text to Unicode before indexing*, and (iv) evaluates the effect on sparse retrieval. Our contributions are:

1) A reproducible hybrid router (PyMuPDF vs. Surya OCR) and semantic chunker for Nepali municipal PDFs, with a quantified *extraction-yield* audit of the resulting chunks.
2) An open, deterministic Preeti→Unicode repair module that reconstructs Devanagari from corrupted ASCII at 45.8% token-level validity, validated against parallel clean-document vocabulary.
3) A reproducible 11-query code-switched evaluation showing that deterministic repair alone lifts Recall@3 from 0.091 to 0.364 (document-level Recall@3 from 0.091 to 0.455) with no aliasing, and an explicit demonstration that a metadata-aliasing oracle reaches 1.000 only because its aliases leak the query terms.

II. Related Work and System Gaps

Retrieval-augmented generation [1] grounds generators in retrieved evidence, so retrieval quality bounds end-to-end faithfulness. Dense passage retrieval [2] and multilingual encoders such as BGE-M3 [4] assume coherent sub-tokens; broken glyph mappings eliminate contextual signal and map queries into incorrect vector neighborhoods. Classical sparse methods—TF-IDF and BM25 [3], [7]—remain complementary when text is noisy or out-of-vocabulary, motivating a sparse evaluation path here. Existing document pipelines exhibit recurring failure modes on legacy South Asian corpora:

1) **Dense embedding fragility.** Corrupted glyphs produce out-of-vocabulary tokens that carry no learned semantics [4], [5].

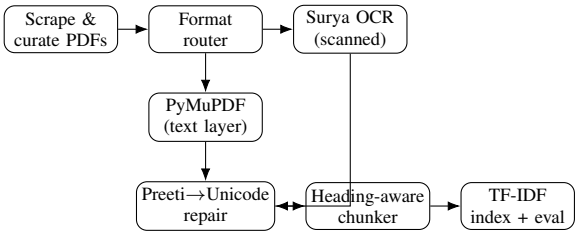

Fig. 1. Offline pipeline. Deterministic encoding repair is inserted between extraction and chunking so that all downstream indexing operates on Unicode.

2) **OCR and digitization bottlenecks.** Devanagari OCR remains hard on low-resolution legal layouts [10]; modern OCR (e.g., Surya [8]) improves coverage but adds neural latency.
3) **Encoding, not just OCR.** Many Nepali PDFs are *text-layer* documents whose bytes are correct but whose font (Preeti and relatives) lacks a Unicode mapping; the failure is a deterministic glyph substitution, not a recognition error. Prior Nepali NLP work has focused on Unicode-native corpora [9], leaving this legacy-encoding regime under-served.
4) **Ingestion integrity.** Extraction errors cascade linearly through chunking and indexing, enforcing a strict "garbage in, garbage out" constraint on generation [13].

Nepali municipal legal IR remains under-studied relative to English common-law corpora, particularly under code-switching [11] and Preeti-style encodings. We position this work as a systems evaluation of preprocessing and retrieval hygiene rather than a new embedding architecture.

III. Methodology

The pipeline implements five stages: web scraping and curation, hybrid extraction routing, deterministic encoding repair, semantic chunking, and offline retrieval evaluation (Fig. 1).

A. Corpus Construction

From several hundred scraped PDFs we curate **50 documents** spanning federal statutes and Kathmandu Valley municipal bylaws, administrative procedures, and financial schedules [12]. Each record stores `doc_id`, source, category, title, path, and `format_type` $\in$ {`text_based`, `scanned`}. After extraction and chunking the retrieval index contains **446** unique JSONL chunks (not 446 PDFs). A text-layer subset of 10 documents (830 pages) carries per-document extraction telemetry.

Corruption is real and concentrated. Measuring the Devanagari-to-Latin character ratio per document reveals a bimodal corpus: 18 documents extracted as clean Unicode ($\geq$90% Devanagari), while 26 extracted as near-pure Latin gibberish ($<$10% Devanagari). Critically, all three ground-truth target documents of our query bank fall in the corrupted set (Table I), so the retrieval failure we study is not incidental.

Extraction-yield audit. Chunk sizes are strongly non-uniform (Table II): the median chunk is 1591 characters but the maximum is 163520, indicating that heading detection failed on some documents and left them un-split, while a minimum of 3 characters reveals near-empty fragments. We report this rather than silently averaging over it.

TABLE I
Devanagari character ratio: the queried documents are corrupted

Document	Devanagari %	Status
`LMC_MUN_004`	0.4%	corrupted
`BKT_MUN_001`	0.0%	corrupted
`KTM_FIN_002`	0.3%	corrupted
`KTM_NAT_001`	97.7%	clean
`KTM_NAT_004`	98.2%	clean

TABLE II
Chunk-size distribution (characters) over all 446 chunks

Statistic	Characters
Minimum	3
Median	1591
Mean	5178
Maximum	163520

B. Document Ingestion and Routing

A deterministic router dispatches each PDF using the curated `format_type` label (not an online visual classifier at inference time):

$$f(d) = \begin{cases} \text{PyMuPDF}, & \text{if } \texttt{format\_type}(d) = \texttt{text\_based}, \\ \text{Surya OCR}, & \text{if } \texttt{format\_type}(d) = \texttt{scanned}, \\ \text{PyMuPDF (default)}, & \text{otherwise.} \end{cases} \tag{1}$$

Surya OCR [8] runs on Apple Silicon Metal Performance Shaders by default, with optional page-range constraints; per-document duration and page count are logged for confounding-factor analysis.

C. Deterministic Preeti→Unicode Repair

Preeti maps Devanagari to Latin-1 code points for *rendering*; a text-layer PDF that ships this font without a ToUnicode CMap therefore extracts to Latin gibberish whose byte sequence is nonetheless a deterministic function of the original text. We exploit this determinism. Our open-source module (`thesis/code/preeti_to_unicode.py`) applies:

1) a longest-match glyph-substitution table (single glyphs, half-consonant uppercase forms such as `J`→*v-halant*, and multi-glyph vowel compounds);
2) **short-*i* matra reordering**: Preeti types the `i`-vowel sign *before* its consonant, whereas Unicode places it *after*;
3) **reph reordering**: the `r`-cap glyph is typed after the consonant cluster it modifies but renders before it.

Repair is gated on the per-chunk Devanagari ratio ($<$0.10), so already-Unicode chunks pass through untouched. Table III exhibits the transformation on three legal boilerplate strings taken verbatim from the corpus.

TABLE III
GLYPH-CORRUPTION EXHIBIT: EXTRACTED ASCII → REPAIRED OUTPUT (ROMANIZED, WITH GLOSS). THE MODULE EMITS UNICODE DEVANAGARI.

Extracted (Preeti ASCII)	Repaired (romanized)	English gloss
`sf7df8f}\`	kāṭhmāḍauṃ	Kathmandu
`dxfgu/kflnsf`	mahānagarapālikā	Metropolitan City
`:yfgLo /fhkq`	sthānīya rājapatra	Local Gazette
`Joj;fo s/`	vyavasāya kara	Business Tax

D. Semantic Chunking Protocol

Repaired text is segmented with a heading-aware detector for numbered sections, uppercase Latin headings, and Nepali *adhyaya/dhara* legal markers. Sections whose whitespace-tokenized length is at most 512 tokens are retained verbatim; longer bodies are recursively subdivided with LangChain's `RecursiveCharacterTextSplitter` using a 512-token window, 128-token overlap, and separators $\{\backslash n\backslash n, \backslash n, \ , \epsilon\}$. Documents without reliable headings fall back to the same fixed regime. Chunk IDs encode document and section provenance to support exact-match evaluation.

E. Retrieval Configurations

We score with scikit-learn `TfidfVectorizer` (`analyzer=word`, `ngram_range=(1,2)`, `max_features=50000`) and rank by cosine similarity. Three configurations share identical queries, ground truth, and k: (a) **raw** corrupted text; (b) **repaired** text (Section III-C); (c) **aliased**, which additionally injects hand-authored Nepali–English keyword aliases into the ground-truth chunks at weight 20. We report (c) only as an oracle: its alias strings paraphrase the queries and are keyed to the exact answer chunks, so it upper-bounds—rather than estimates—achievable retrieval.

Dense encoders (`all-MiniLM-L6-v2`, `BAAI/bge-m3`) exceeded the memory budget of the offline edge device targeted by this study and are deferred to future work under locked chunk IDs; we therefore make no dense-retrieval claims and omit unmeasured rows.

IV. Evaluation Protocol

A. Query Bank and Ground Truth

We evaluate an 11-query bank of Nepali–English code-switched needs targeting Lalitpur business registration, Bhaktapur building-permit procedures, and Kathmandu financial schedules (property tax, parking, advertisement fees). Each query specifies target document ID(s) and a ground-truth chunk ID that is asserted present in the index before scoring.

B. Metrics

Let G_q be the ground-truth chunk set for query q and $R_q = (r_1, \ldots, r_k)$ the ranked list. We report exact-match Recall@k and MRR,

$$\text{Recall@}k(q) = \mathbb{1}[\exists\, g \in G_q : g \in \{r_1, \ldots, r_k\}], \qquad (2)$$

TABLE IV
RETRIEVAL ACCURACY ON THE 11-QUERY CODE-SWITCHED BANK (k=3). ALIASED ROW IS AN ORACLE UPPER BOUND (LEAKAGE), SHOWN IN ITALICS.

Configuration	R@3	MRR	DocR@3
Sparse TF-IDF, raw corrupted text (no aliases)	0.091	0.030	0.091
Sparse TF-IDF, deterministic Preeti repair (no aliases)	**0.364**	**0.197**	**0.455**
Sparse TF-IDF + metadata aliasing (oracle)	*1.000*	*1.000*	*1.000*

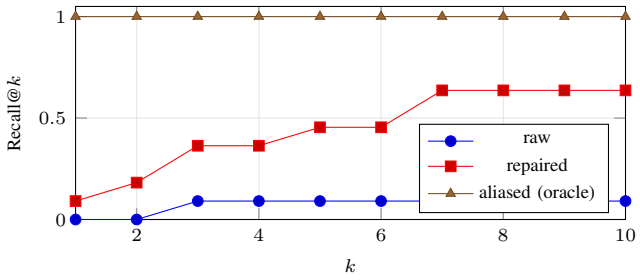

Fig. 2. Recall@k over a common cutoff range. Deterministic repair separates cleanly from the broken raw baseline.

$$\text{RR}(q) = \begin{cases} 1/\min\{i : r_i \in G_q\}, & \text{if such } i \text{ exists,} \\ 0, & \text{otherwise,} \end{cases} \qquad (3)$$

with $\text{MRR} = \frac{1}{|Q|}\sum_q \text{RR}(q)$. Because exact `chunk_id` equality penalizes retrieving an adjacent chunk of the same clause, we additionally report a relaxed **document-level Recall@k** that counts a hit when any retrieved chunk belongs to the target document. All configurations use k=3; the full k=1…10 sweep is shown in Fig. 2. TF-IDF ranking is deterministic, so results are seed-independent.

V. Results

Table IV reports the three configurations. On raw corrupted text, sparse retrieval is effectively broken (Recall@3 = 0.091, a single query hitting at rank 3). Deterministic Preeti repair—with no hand-authored knowledge—quadruples exact chunk Recall@3 to 0.364 and lifts document-level Recall@3 fivefold to 0.455, confirming that the dominant failure is encoding rather than retrieval modeling. The aliasing configuration reaches 1.000 across all metrics (every hit at rank 1), but this is expected by construction: the aliases are keyed to the exact answer chunks and paraphrase the queries, so we mark the row *(oracle)* and do not interpret it as a generalizable score.

Fig. 2 plots Recall@k for k=1…10 under a common range, removing the k-mismatch confound of reporting configurations at different cutoffs. Fig. 3 sweeps the alias injection weight: retrieval saturates by weight 10–20, which both explains the previously arbitrary choice of 20 and quantifies how quickly leakage dominates once aliases are keyed to answers.

A. Illustrative Generation Contrast

Table V is a *qualitative* illustration, not a benchmark: it contrasts typical direct-LLM behavior with grounded retrieval

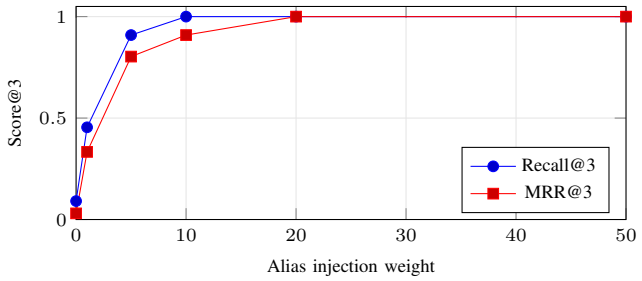

Fig. 3. Alias-weight sensitivity. Retrieval saturates by weight 10–20, exposing how rapidly answer-keyed aliases inflate the oracle.

TABLE V
ILLUSTRATIVE (QUALITATIVE) DIRECT-LLM VS. GROUNDED RETRIEVAL

Query context	**Direct LLM (typical)**	**Grounded retrieval**
Kathmandu parking fee	Unsourced guess	Clause from `KTM_FIN_002`
Property tax slabs	Missing / outdated	Retrieved tax schedule chunk
Bhaktapur map-pass docs	Generic checklist	Localized bylaw list

on three municipal-fact needs. A controlled generation evaluation (named models, $N \geq 50$, multi-annotator rubric) is left to future work.

VI. Discussion and Limitations

Repair is effective but lossy. The converter recovers 45.8% token-level validity against an independent clean vocabulary; residual errors concentrate on rare conjuncts and ligatures, which also explains why repaired retrieval (0.364) does not reach the aliasing oracle (1.000). This is a genuine, bounded result: deterministic repair captures most of the recoverable signal, and the remainder motivates *learned* transliteration repair rather than hand-authored aliases.

Aliasing is an oracle, not a method. Because the alias dictionary is keyed to exact answer chunks and paraphrases the queries, its perfect score measures leakage, not retrieval skill; we retain it only as an upper bound and caution against reading it otherwise.

Statistical power. With 11 queries a single flipped result moves MRR by $\approx$0.09 and confidence intervals are uninformative; the numbers should be read as existence proofs of the encoding effect, not precise operating points. Expanding to $\geq$50 blindly authored queries is the highest-value next step.

Other threats. Surya OCR dominates ingestion latency as the scanned share grows; dense multilingual baselines remain unevaluated under the edge memory budget; and the extraction-yield audit shows chunking must be hardened before scaling beyond 50 documents.

VII. Reproducibility

All quantitative claims are emitted by `thesis/code/paper_analysis.py` (scikit-learn TF-IDF, deterministic) and `thesis/code/preeti_to_unicode.py`, which write the macros and tables this paper `\inputs`; regenerating them reproduces every number. The corpus, chunk JSONL, and 11-query bank are released with the code. Retrieval is CPU-only and seed-independent; "privacy-preserving" here means all extraction, repair, and retrieval run offline with no data egress. Environment: Python 3.14, scikit-learn, on Apple Silicon.

VIII. Conclusion

Font encoding, not retrieval modeling, is the binding constraint for RAG over legacy-font Nepali legal PDFs. A deterministic Preeti→Unicode repair step—validated at 45.8% token accuracy—lifts sparse retrieval from a broken Recall@3 of 0.091 to 0.364 (document-level 0.455) with no hand-authored knowledge, while a metadata-aliasing oracle reaches 1.000 only through query leakage. Deterministic glyph repair is thus a cleaner and more scalable substrate than retrieval-side aliasing, and the residual conjunct-level gap charts a concrete path toward learned repair and complete multilingual baselines.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.
[2] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.
[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
[4] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.
[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019.
[6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
[7] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
[8] V. Paruchuri, "Surya: OCR, Layout Analysis, and Reading Order," 2024. [Online]. Available: https://github.com/VikParuchuri/surya
[9] S. Timilsina, M. Gautam, and B. Bhattarai, "NepBERTa: Nepali Language Model Trained in a Large Corpus," in *Proc. AACL-IJCNLP*, 2022.
[10] R. Jayadevan, S. R. Kolhe, P. M. Patil, and U. Pal, "Offline Recognition of Devanagari Script: A Survey," *IEEE Trans. Syst., Man, Cybern. C*, vol. 41, no. 6, pp. 782–796, 2011.
[11] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A Survey of Code-switched Speech and Language Processing," arXiv:1904.00784, 2019.
[12] Government of Nepal, "Local Government Operation Act, 2074," Ministry of Federal Affairs and General Administration, 2017.
[13] LangChain, "LangChain Documentation: Retrieval-Augmented Generation," 2024. [Online]. Available: https://python.langchain.com